

Caso de estudio:

Desarrollo de la app móvil LatinoLink.

Marketplace de servicios en español para Toronto.

Ingeniero de software: Carlos López

App publicada: [App Store](#) · [Google Play](#)

Periodo: Febrero – Agosto de 2026

Stack: FlutterFlow, Flutter/Dart,
Firebase, Cloud Functions, RevenueCat

Contacto:

carloslopezr29@gmail.com

linkedin.com/in/carloslopezdev

Cliente

LatinoLink, un negocio con sede en Toronto que encargó un marketplace para conectar a la comunidad latinoamericana de la ciudad con proveedores de servicios de confianza.

Problema

Un marketplace basado en la confianza es tan creíble como lo sea el cumplimiento de sus reglas. Como la app escribe directamente en la base de datos, cada regla de la que depende el negocio, desde los saldos de créditos hasta los pagos y la verificación, tenía que quedar protegida donde los usuarios no pudieran alcanzarla.

Resultados

Desarrollo integral de los requisitos a producción en siete meses

Arquitectura completa del sistema y modelo de datos

Seguridad aplicada en el servidor, con una suite adversarial de 81 pruebas

Motor de emparejamiento de leads por distancia, con recuperación automática

Pagos idempotentes dentro de la app con RevenueCat

Lanzamiento en el App Store y Google Play, que hoy atiende a más de 100 usuarios

Índice

1. Introducción
2. Visión general del sistema
 - 2.1 Roles de usuario y modelo de créditos
 - 2.2 Arquitectura
 - 2.3 Dónde se hacen cumplir las reglas
3. Proceso de desarrollo
4. Auditoría de seguridad
 - 4.1 Criterios de auditoría
 - 4.2 Campos de saldo y estado
 - 4.3 Aceptación de leads
 - 4.4 Estado de membresía
 - 4.5 Verificación telefónica
 - 4.6 Hallazgos de la auditoría
5. Motor de asignación de leads
 - 5.1 Cascada vs. difusión
 - 5.2 Manejo de rechazos y recuperación
6. Requisitos del App Store
 - 6.1 Cómo App Review moldeó el diseño
 - 6.2 Procesamiento idempotente de pagos
7. Lecciones y conclusión
 - 7.1 Qué haría diferente
 - 7.2 Conclusión
- Referencias

1. Introducción

LatinoLink es un marketplace de servicios que prioriza el español para el Greater Toronto Area (GTA). Lo diseñé y construí como único ingeniero para una clienta, llevándolo desde los primeros cuestionarios de descubrimiento hasta producción.

La idea partió de una observación simple: los latinoamericanos del GTA ya encuentran plomeros y electricistas a través de grupos de Facebook y cadenas de WhatsApp, pero esas redes no pueden verificar a un proveedor ni avalar la calidad de su trabajo. Una barrera lingüística empeora ambas cosas. Explicar un trabajo técnico en un segundo idioma es difícil, y el idioma es el motivo de discriminación reportado con mayor frecuencia entre los latinoamericanos en Canadá, por encima de la etnia y la raza [1]. Un marketplace que reemplaza estas redes en realidad vende confianza, así que LatinoLink verifica a los usuarios por SMS y les pide a los proveedores pruebas de quiénes son: licencias de Ontario, seguros, certificaciones y registro de negocio.

Mi propósito aquí es mostrar cómo se hace cumplir esa confianza en lugar de prometerla. LatinoLink corre sobre Firebase, un backend serverless elegido para lanzar rápido: la app lee y escribe directamente en la base de datos, sin un servidor de aplicaciones propio. Esa decisión lleva el cumplimiento de las reglas a dos capas: reglas de seguridad que protegen cada solicitud y Cloud Functions que se encargan de todo lo privilegiado. Cada regla de la que depende el negocio o bien se ejecuta en una de esas capas, o bien puede evadirla cualquiera con la app y un poco de curiosidad. Eso incluye si un proveedor puede cambiar su propio saldo de créditos, quién puede aceptar un lead, si un pago se otorga exactamente una vez y quién aparece en el directorio. Decidir dónde vive cada regla fue el problema de ingeniería central del proyecto, y es el lente que este estudio aplica de principio a fin.

El estudio comienza con el sistema en sí: los roles de usuario, la arquitectura y los tres lugares donde puede ejecutarse una regla, seguido del proceso de desarrollo. En el centro del estudio está una auditoría de seguridad que realicé sobre mis propias reglas después del lanzamiento, en la que tres controles se sostuvieron y el cuarto sumó una verificación más del lado del servidor. A partir de ahí, aborda el motor de asignación de leads y los cambios que App Review trajo a los pagos y a la verificación, y cierra con lo que construiría de otra manera.

2. Visión general del sistema

2.1 Roles de usuario y modelo de créditos

LatinoLink tiene tres tipos de usuario.

Los clientes pueden explorar un directorio de negocios latinos y una lista de servicios de proveedores. Verifican su número de teléfono con un código SMS cuando envían su primera solicitud de servicio, y los proveedores lo verifican en su primera compra de créditos, los dos puntos donde un número sin verificar empieza a costarle dinero al marketplace. Para limitar el spam, cada cliente puede tener como máximo tres solicitudes activas a la vez.

Los proveedores pueden ofrecer varios servicios. Durante el onboarding, se le pide a cada proveedor que suba su certificación, ya sea una certificación de Ontario para los oficios regulados o una certificación de su país de origen para los no regulados. Los proveedores compran paquetes de créditos mediante compra dentro de la app, y esos créditos se gastan después para aceptar los leads que se les ofrecen dentro del rango de su ubicación.

Los dueños de negocios suben la información de su negocio y envían una ficha, que un administrador revisa antes de que se publique. La categoría de la ficha se valida contra una taxonomía de membresías; por lo tanto, no se puede usar un oficio para crear una ficha en el directorio. Una vez aprobada, el dueño compra una membresía mensual o anual para aparecer destacado en el directorio que ven los clientes.

Modelo de créditos. La solicitud de servicio de un cliente se convierte en un lead, que recorre el sistema en una sola pasada. El servidor geocodifica el código postal y ordena a los proveedores elegibles por distancia de Haversine, y el más cercano recibe una única oferta con una ventana de treinta minutos. Aceptar descuenta un crédito y cierra el lead; rechazarlo o dejar que se agote el tiempo lo pasa al siguiente candidato. Nunca hay más de una oferta pendiente, y un sweeper programado recoge cualquier lead que se haya quedado sin ella.

Los créditos son la vía de ingresos del lado de los proveedores: se compran en paquetes de tres mediante compra dentro de la app, se guardan como saldo en el registro del usuario y se gastan a razón de un crédito por lead aceptado. Las membresías son la vía de ingresos del lado de los negocios.

2.2 Arquitectura

El cliente es una app de FlutterFlow construida con componentes sobre un patrón MVVM. El backend es serverless y tiene dos capas: Firebase (Firestore, Auth y Storage) como la capa de backend como servicio que la app lee directamente, y Cloud Functions para las escrituras privilegiadas y las integraciones con terceros. El backend está orientado a eventos y funciona con triggers de Firestore, un programador de tareas y webhooks verificados.

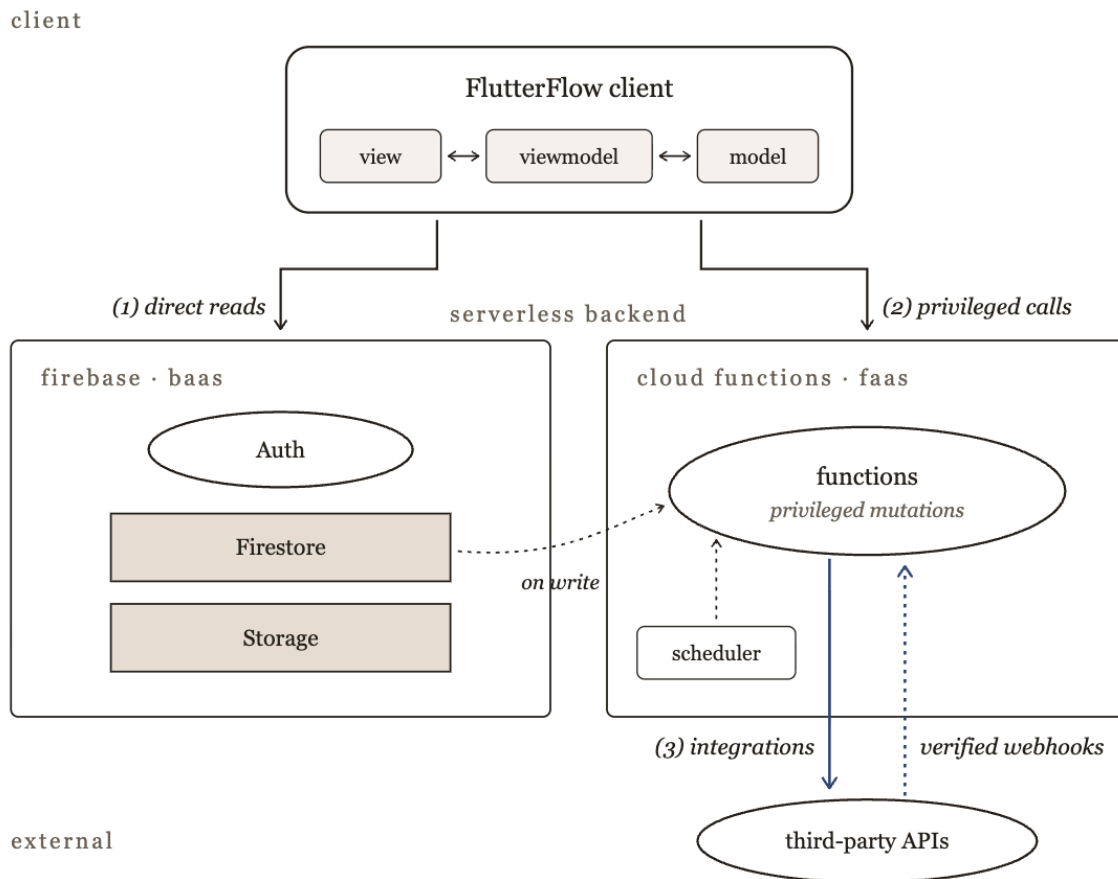


Figura 2: Arquitectura del sistema

2.3 Dónde se hacen cumplir las reglas

Toda regla que este sistema declara se ejecuta en una de tres ubicaciones, y las propiedades de cada ubicación son distintas:

1. **Reglas de seguridad.** Firestore evalúa las reglas en cada solicitud, y la capa de reglas no es alcanzable desde el cliente. Por defecto, opera a nivel de documentos y colecciones. Sin embargo, sobre ese comportamiento por defecto se escribieron operadores

personalizados de inmutabilidad de campos para llegar a una granularidad a nivel de campo, que es lo que impide que valores sensibles como el saldo de créditos, el estado de la membresía y las banderas de verificación sean modificados por su propio dueño.

2. **Cloud Functions.** Las funciones corren en infraestructura que el cliente no controla y contienen la lógica privilegiada: asignación de leads, manejo de compras dentro de la app, y creación y eliminación de cuentas. Responden a triggers de la base de datos, solicitudes callable y webhooks verificados. Sin embargo, una función solo hace cumplir una regla si el cliente no puede saltársela, y lo que la vuelve imposible de saltar es una regla de seguridad que deniega la escritura directa. Las dos capas dependen una de la otra.
3. **La app.** El cliente de FlutterFlow consulta el estado y restringe la interfaz en consecuencia, y las acciones personalizadas escritas en Dart invocan las Cloud Functions privilegiadas donde el constructor visual no alcanzaba.

Una regla también puede ser declarada por el producto sin ejecutarse en ningún lugar del sistema, en cuyo caso depende por completo de que los usuarios decidan cumplirla.

Esta distinción aporta el criterio que aplica el resto del estudio. Un control se sostiene si se ejecuta en las dos primeras ubicaciones, y no se sostiene si se ejecuta solo en la tercera o en ninguna.

3. Proceso de desarrollo

El proyecto se desarrolló en tres fases, cada una alimentando a la siguiente: el descubrimiento convirtió la idea de la fundadora en requerimientos, la iteración convirtió los requerimientos en un producto funcional y la verificación comprobó que el producto hiciera cumplir lo que prometía.

Descubrimiento. La idea de la fundadora se convirtió en una lista de requerimientos mediante cuestionarios estructurados y diagramas de flujo, completados en tres pasadas. El ciclo demostró su valor desde temprano: el requerimiento de certificación se invirtió entre la segunda y la tercera pasada, antes de que se construyera cualquier parte de él.

Iteración. El desarrollo siguió un modelo de desarrollo rápido de aplicaciones con una capa ágil encima, donde cada prototipo se construía para ser revisado, no defendido. Figma AI y Claude Design generaron los mockups, pero ninguno salió sin revisión, ya que ninguna de las

dos herramientas produce el mismo resultado correcto dos veces. Cada entregable iba directo a la fundadora, y su retroalimentación definía el alcance de la siguiente iteración. Las herramientas aceleraron la producción; la línea directa con la fundadora la mantuvo apuntando a los requerimientos correctos.

Verificación. Las pruebas apuntaron a la frontera de autorización, no solo al camino feliz: la suite del emulador comprueba que se deniega el acceso no autorizado y la manipulación, y los flujos de pago se probaron de extremo a extremo en TestFlight y en el canal de pruebas internas de Play, desde la compra hasta la eliminación, el nuevo registro y la restauración. Después del lanzamiento, el mismo estándar se aplicó al propio sistema, que es la revisión que reporta la Sección 5.

4. Auditoría de seguridad

Después del lanzamiento, con más de 100 usuarios en producción, regresé a comprobar si los controles realmente se sostenían. Elegí los cuatro que protegen el dinero y la integridad del marketplace: saldos de créditos, aceptación de leads, estado de membresía y verificación por SMS. Cada uno conforme al estándar establecido en la Sección 2.3: solo cuenta si se ejecuta en algún lugar que el cliente no puede alcanzar. Dos registros hicieron posible esa revisión: la justificación escrita dentro de las propias reglas y funciones durante el desarrollo, incluidas las alternativas descartadas, y una suite de pruebas en el emulador construida para atacar la frontera de autorización, con 63 de sus 81 pruebas verificando que se deniegan el acceso no autorizado o la manipulación. Tres controles se sostuvieron. Uno no, y el resto de esta sección cubre cómo salió a la luz esa brecha y cómo se cerró.

4.1 Criterios de auditoría

Cada control se somete a dos criterios. El primero es la ubicación, ya establecida en la Sección 2.3: un control debe ejecutarse en las reglas de seguridad o en una Cloud Function por la que las reglas obligan a pasar al cliente. Todo lo que se deja en manos del cliente termina como trabajo manual, y ese trabajo crece con cada usuario que se incorpora.

El segundo es la selección. Bloquear todos los campos es un fallo tan grave como no bloquear ninguno, porque las ediciones ordinarias se vuelven imposibles; los campos que tienen consecuencias se bloquean y el resto queda abierto. Un control se sostiene solo si cumple ambos.

Cada control a continuación respalda una promesa que hace LatinoLink: ningún usuario puede alterar su propio saldo, rol o estado; solo el proveedor al que se le ofreció un lead puede aceptarlo; solo los negocios que pagan aparecen en el directorio; y ningún número sin verificar puede solicitar un servicio ni comprar créditos. Cada uno se evalúa contra ambos criterios.

4.2 Campos de saldo y estado

Promesa. Ningún usuario puede alterar su propio estado económico o de verificación: un proveedor no puede tocar su saldo de créditos ni el estado de su certificado, un dueño de negocio no puede cambiar el nivel de su membresía ni su estado de destacado, y un cliente no puede editar una solicitud una vez creada.

Cumplimiento. Las reglas de Firestore operan sobre documentos completos y no tienen noción de los campos que contienen, así que la protección a nivel de campo tuvo que construirse encima. Una función auxiliar personalizada y null-safe compara cada campo bloqueado antes y después de una escritura y rechaza cualquier cambio: seis campos en los perfiles de usuario, el estado del certificado en los perfiles de proveedor y diez campos de suscripción en las fichas de negocios. Tres de los campos de usuario también se bloquean en la creación y, en su lugar, los inicializa un trigger del servidor, de modo que el bloqueo aplica desde la primera escritura. Los leads se saltan por completo el tratamiento a nivel de campo, ya que todas las actualizaciones del cliente se deniegan de entrada. La suite del emulador cubre estas rutas con 63 aserciones de denegación. Quince campos del perfil, entre ellos el nombre visible y las preferencias de notificación, se mantienen abiertos a propósito.

Resultado. Se sostiene en ambos criterios: se hace cumplir del lado del servidor y se bloquea solo donde un cambio tiene consecuencias.

4.3 Aceptación de leads

Promesa. Un lead se ofrece a un proveedor a la vez, y solo ese proveedor puede aceptarlo.

Cumplimiento. Los leads son inmutables en la capa de reglas, así que la aceptación no puede escribirse directamente y debe pasar por una función callable. Antes de escribir nada, la función verifica cuatro condiciones: quien llama es el proveedor actual del lead, el lead sigue asignado, la oferta está pendiente y no ha expirado, y quien llama tiene al menos un crédito.

Después actualiza el lead, el usuario y la asignación en una sola transacción, de modo que los tres tienen éxito o fallan juntos. El saldo se verifica dos veces, por razones distintas: una durante el emparejamiento, como un filtro rápido que descarta a los proveedores que claramente no pueden pagar, y otra al momento del cobro, de forma transaccional, para que las ofertas concurrentes nunca puedan llevar un saldo por debajo de cero.

Aquí es donde las dos capas del servidor dependen una de la otra. Las reglas no pueden expresar una lectura transaccional a través de tres documentos, y una función no garantiza nada si el cliente puede evadirla; las reglas deniegan la escritura directa, y esa denegación vuelve ineludibles las verificaciones de la función. Las reglas también cargan con su propia parte: bloquean estado de servidor fabricado en la creación de leads, y limitan la visibilidad de un lead a su cliente, a su proveedor actual y a los proveedores a los que se les ofreció antes.

Resultado. Se sostiene: cada capa hace cumplir lo que la otra no puede, y ninguna puede evadirse desde el cliente.

4.4 Estado de membresía

Promesa. Solo los miembros que pagan aparecen destacados en el directorio.

Cumplimiento. La membresía vive en tres lugares: el recibo de la tienda, el entitlement en RevenueCat y una bandera en caché en Firestore que el directorio lee por velocidad. El cliente no puede escribir esa bandera: los campos de membresía están bloqueados, y el webhook de RevenueCat es el único que los escribe. Un campo se mantiene abierto a propósito, ya que el cliente lo establece justo antes de una compra, y la regla deja constancia de por qué.

A diferencia de los otros controles, este también gobierna las lecturas. Con el mismo dato guardado en tres lugares, la pregunta no es solo quién puede cambiarlo, sino también en qué copia confiar. La caché nunca es autoritativa: cuando marca negativo, la app consulta el entitlement directamente y falla en modo cerrado si esa consulta da error. Un miembro que paga puede ver brevemente una función bloqueada, pero uno que no paga nunca ve una desbloqueada.

Resultado. Se sostiene, aceptando una breve ventana de negativos desactualizados a cambio de cero falsos positivos.

4.5 Verificación telefónica

Promesa. Un cliente se verifica por SMS antes de solicitar un servicio, y un proveedor, antes de comprar créditos.

Cumplimiento. La revisión de apps exige que este tipo de datos se recopile solo cuando una función lo necesita, así que la verificación ocurre en la primera solicitud y en la primera compra, no en el registro. La bandera en sí es infalsificable: está bloqueada contra escrituras del cliente y solo la establece la función de verificación, que compara el código enviado con un hash almacenado, sujeta a un límite de intentos, una expiración y límites de frecuencia. El número solo se guarda cuando la verificación tiene éxito.

Al correr revisiones de seguridad contra las reglas después del lanzamiento, descubrí que la barrera solo se hacía cumplir en la interfaz de la app; nada en el servidor leía la bandera. El valor estaba protegido, pero su uso no; un valor que nada consume es inerte. La corrección se hizo en la regla de creación de leads: esta ya lee el documento de usuario de quien llama para confirmar su rol, así que la verificación reutiliza esa misma lectura sin costo adicional. Dos nuevas pruebas en el emulador la cubren, lo que lleva la suite a 81 pruebas y 63 aserciones de denegación.

Resultado. Se hace cumplir del lado del servidor. Bloquear un campo y usarlo como barrera son decisiones distintas, y ahora cada campo bloqueado tiene un lector del lado del servidor.

4.6 Hallazgos de la auditoría

Todos los controles que protegen dinero o identidad se sostuvieron: saldos de créditos, aceptación de leads y estado de membresía. La verificación fue la excepción, y la razón es instructiva. El criterio para decidir qué bloquear había sido la consecuencia económica, y la bandera de verificación tiene consecuencias sin involucrar dinero: gobierna la elegibilidad, es decir, si un lead proviene de un cliente verificado. Se bloqueó con el mismo rigor que un saldo, pero bloquear un campo y usarlo como barrera son decisiones distintas, y solo la primera se había tomado en el servidor.

Esa distinción ahora es una invariante de diseño: todo campo bloqueado tiene un lector del lado del servidor, y todo campo que lee una verificación del lado del servidor está bloqueado.

Como aplica a campos que aún no existen, mantiene alineados el bloqueo y las barreras a medida que el sistema crece.

Ninguna de estas reglas viene por defecto en la plataforma. Firestore deniega por defecto, pero no tiene noción de campos, y el constructor de reglas de FlutterFlow no podía expresar seis de las doce colecciones, así que cada regla se escribió a mano y cada campo se eligió deliberadamente. El saldo de créditos y el nombre visible están en el mismo documento, bajo la misma regla; solo uno está bloqueado. El resultado es un sistema cuya seguridad sigue a su economía: toda ruta que mueve dinero o da acceso a él se hace cumplir en el servidor y está respaldada por una suite de pruebas adversariales.

5. Motor de asignación de leads

El motor de asignación de leads carga con la primera fuente de ingresos de la app. Cuando un cliente solicita un servicio, una Cloud Function ejecuta el matcher, que ordena a los proveedores calificados por distancia de Haversine. Las consultas por geohash en Firestore eran la alternativa, pero a la escala actual, calcular la distancia para cada proveedor calificado mantiene el matcher simple y fácil de razonar; los índices geoespaciales de Redis son el camino a seguir cuando la base de proveedores lo rebase.

1. El código postal del cliente se geocodifica contra una caché en Firestore, con la Google Geocoding API como respaldo cuando no está en caché. Las coordenadas de los proveedores se precálculan en el registro, así que en la ruta de la solicitud no se geocodifica a ningún proveedor.
2. Dos consultas encuentran a los proveedores calificados: una consulta de grupo de colecciones para quienes ofrecen el servicio solicitado y, después, una búsqueda de sus registros de usuario. El proveedor más cercano recibe una única oferta pendiente con una ventana de treinta minutos.
3. Al aceptar, una transacción descuenta el crédito, marca la oferta como aceptada y registra al proveedor en el lead. Ante un rechazo o una expiración, el matcher pasa al siguiente candidato.

5.1 Cascada vs. difusión

Se evaluaron dos modelos: difundir cada lead a todos los proveedores cercanos y dejar que gane el primero en reclamarlo, u ofrecerlo a un proveedor a la vez en orden de distancia. El requerimiento de la fundadora lo decidió: cada proveedor tiene treinta minutos para aceptar o rechazar antes de que el lead siga adelante, y una ventana por proveedor solo funciona si un solo proveedor tiene el lead a la vez. Así que elegí la cascada. También es el modelo más robusto. La difusión multiplica la carga de la base de datos con cada lead y crea una condición de carrera en la aceptación, dejando a cada proveedor perdedor con una oferta obsoleta que conciliar. La cascada mantiene exactamente una oferta pendiente por lead, así que aceptar o rechazar es una actualización de un solo registro en una transacción, y un rechazo solo implica encontrar al siguiente candidato. También les da a los proveedores lo que esperan: la coincidencia más cercana siempre se ofrece primero.

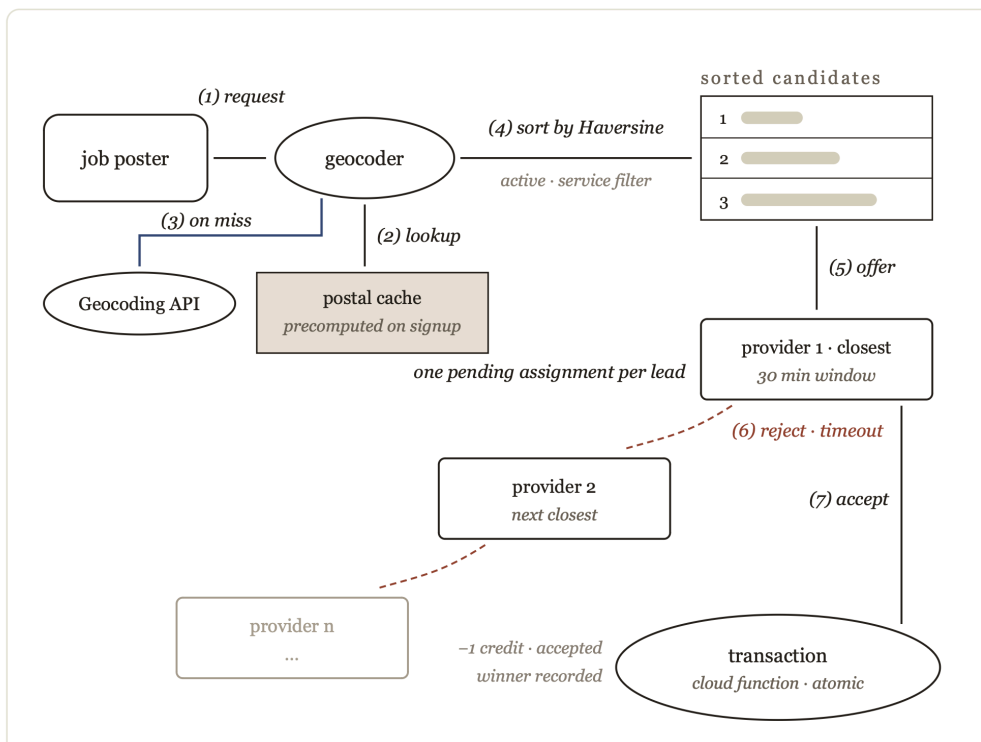


Figura 3: Ciclo de vida de un lead a través del modelo en cascada.

5.2 Manejo de rechazos y recuperación

Rechazar un lead dispara la búsqueda del siguiente proveedor, y esas consultas no tienen cabida dentro de una transacción, así que el rechazo se ejecuta en dos fases. La Fase A es

transaccional: confirma que quien llama es el proveedor actual y que la oferta sigue pendiente, registra el rechazo y limpia el puntero de proveedor del lead con una marca de tiempo. Después, la Fase B ejecuta el matcher fuera de la transacción.

Todo lo que ocurre después de la Fase A es recuperable por diseño. Una Fase B interrumpida deja exactamente un estado detectable: un lead marcado como asignado, sin proveedor actual, durante más de cinco minutos. Un sweeper que corre cada dos minutos encuentra ese estado y vuelve a ejecutar el matcher. La marca de tiempo existe solo para ese propósito, y garantiza que ningún lead se quede nunca varado sin una oferta.

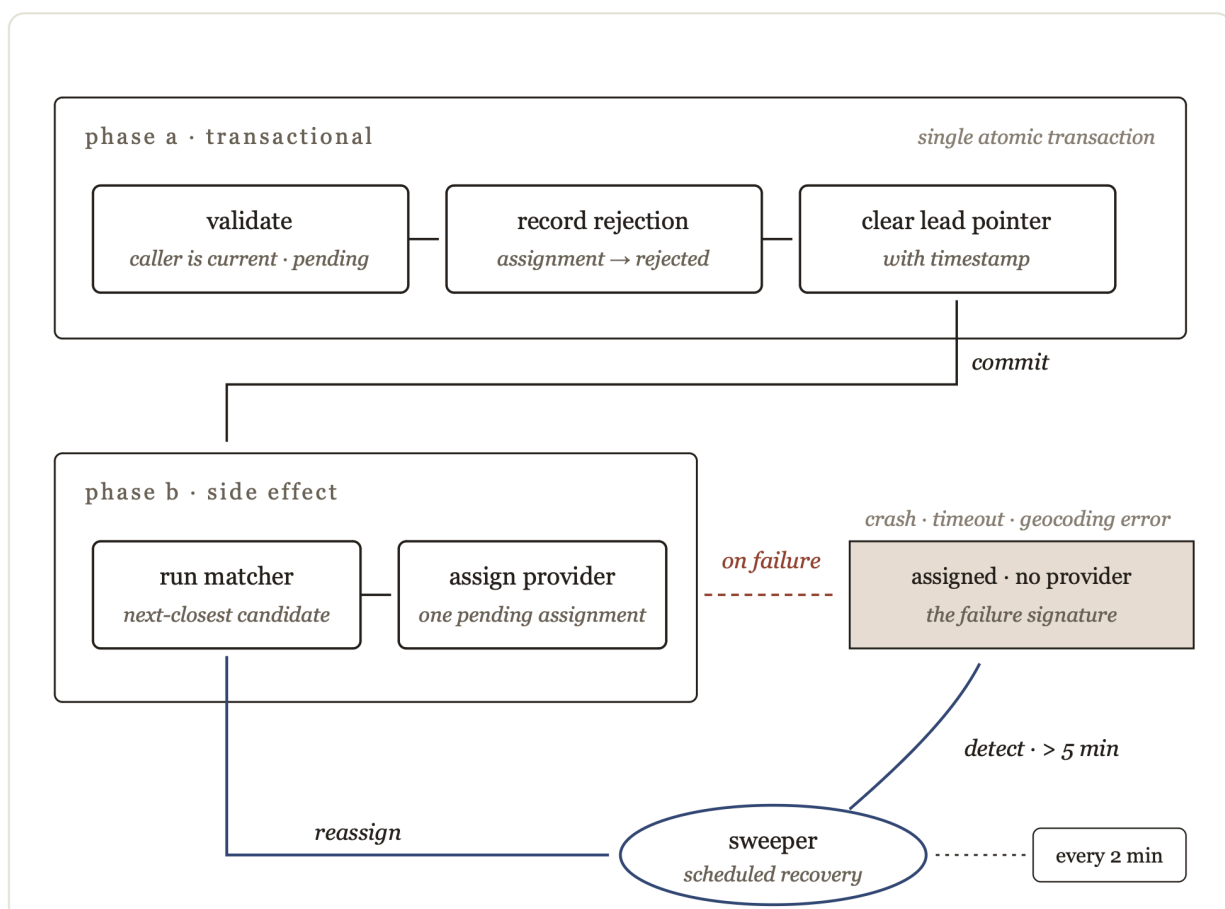


Figura 4. Sweeper y rechazo en dos fases.

6. Requisitos del App Store

Las decisiones de diseño vinieron de tres fuentes: los requerimientos de la fundadora, mis propias decisiones de ingeniería y App Review. Dos de las directrices de Apple redefinieron el sistema de pagos y el flujo de usuario.

6.1 Cómo App Review moldeó el diseño

Pagos, Directriz 3.1.1. Los bienes digitales que se consumen en una app deben venderse mediante compra dentro de la app. Los créditos pagan servicios que se prestan fuera de la app, lo que era un argumento a favor de una exención, pero la decisión de la fundadora de agregar membresías, una función puramente dentro de la app, zanjó la cuestión. Stripe salió del build, y los pagos pasaron a compra dentro de la app mediante RevenueCat para tener soporte multiplataforma, lo que implicó rediseñar los webhooks y las APIs a su alrededor.

Momento de la verificación, Directriz 5.1.1. Los datos personales solo pueden recopilarse cuando una función los requiere, así que la verificación por SMS pasó del registro a la primera solicitud de un cliente y a la primera compra de créditos de un proveedor. Ahora los usuarios pueden explorar antes de entregar un número de teléfono, y el modelo de datos queda más limpio: los números solo se escriben cuando la verificación tiene éxito, así que la colección de usuarios contiene únicamente números verificados, y eso es lo que le da sentido a la comprobación de unicidad. Una barrera al registrarse habría almacenado un número por cada registro abandonado. La restricción planteó una pregunta que el modelo de amenazas no se había planteado: no si una comprobación puede evadirse, sino por qué se guardan esos datos en primer lugar.

6.2 Procesamiento ‘Idempotent’ de pagos

Todo endpoint que mueve dinero debe ser idempotente. El webhook de RevenueCat entrega cada evento al menos una vez y reintenta hasta cinco veces cuando no llega respuesta, así que un reintento tras una respuesta perdida otorgaría un segundo periodo de membresía por un solo pago. El handler basa la idempotencia en el identificador de evento de RevenueCat y lo reclama con una escritura de solo creación de un documento con ese ID: la primera entrega tiene éxito, y cualquier repetición no logra crear el documento y termina sin actuar.

El identificador de transacción de la tienda es la clave intuitiva, pero falla en silencio. En Google Play, una renovación agrega un sufijo incremental al ID de pedido base, mientras que una expiración no crea un pedido nuevo y reutiliza el último. Una guarda basada en ese identificador trataría la expiración como ya procesada, la omitiría y dejaría la membresía activa indefinidamente. Ese comportamiento se confirmó contra los registros de transacciones de RevenueCat, y el ciclo de vida completo se probó de extremo a extremo en TestFlight y en el canal de pruebas internas de Play: compra, eliminación, nuevo registro y restauración.

7. Lecciones y conclusión

7.1 Qué haría diferente

Revisar las directrices de las tiendas en la etapa de diseño. Diez ciclos de revisión en dos versiones produjeron los cambios más costosos del proyecto, todos arquitectónicos: los pagos pasaron a compra dentro de la app, la verificación salió del registro y la eliminación de cuentas se construyó desde cero. Cada uno podía descubrirse en las directrices publicadas por Apple antes de escribir una sola línea de código. Revisar todo el flujo contra las directrices de ambas tiendas desde el principio cuesta un día y ahorra semanas.

Automatizar las partes de la revisión que no requieren criterio. La revisión de certificaciones tiene que seguir siendo humana; perseguir a los proveedores por documentos faltantes, no. Una función que detecte lo que le falta a un perfil y envíe el seguimiento eliminaría ese trabajo recurrente y dejaría la decisión en manos de una persona.

7.2 Conclusión

LatinoLink se lanzó para cubrir la necesidad de seguridad en un marketplace de servicios digitales, superando la expectativa del cliente de un MVP. Lo que se entregó va más allá: una suite de pruebas adversariales, pagos idempotentes, un algoritmo de asignación de leads basado en proximidad y una auditoría de seguridad posterior al lanzamiento. LatinoLink está disponible en App Store y Google Play, y cumple con el estándar sobre el que fue construido: confianza garantizada por el sistema, no supuesta de sus usuarios.

Referencias

- [1] Statistics Canada, "Study: Portrait of the Latin American populations in Canada," *The Daily*, 29 jun. 2026. [En línea]. Disponible en: <https://www150.statcan.gc.ca/n1/daily-quotidien/260629/dq260629a-eng.htm>